

SE Web Toolkit 설계

내부 엔지니어링 도구(데이터 조회·상태 모니터링·업무 처리)를 위한 디자인 시스템 + Claude Code 플러그인. (v0.6 · 2026-09-14)
목표: 개발 속도, SE 아이덴티티의 일관성 (**양산형이 아닌, 서비스별 개성을 가진 일관성**), 전문가 수준의 UI 품질, 낮은 진입 장벽.

전제(확정): FE 표준 = React + Vite + TypeScript / 기존 서비스 = React 계열 / 백엔드·배포 = 비표준(무관해야 함) / 배포 = Claude Code 플러그인(marketplace).

1. 한 줄 요약

"같은 가족, 다른 형제" — SE 공통 골격(타이포·간격·컴포넌트 형태·AppShell)은 패키지로 고정하고, 서비스마다 아이덴티티(액센트·마크·시그니처 요소·톤)를 **받드**시/다르게 정하게 한다. Claude는 웹 디자이너처럼 **디자인 플랜 → 구현 → 스크린샷 비평 → 수정** 루프를 돌며, 규칙은 린터와 훅이 기계적으로 지킨다.

핵심 결정 4가지

- 아이덴티티를 2층으로 나눈다. 브랜드 코어(고정) / 서비스 아이덴티티(가변, 필수). 개성은 옵션이 아니라 필수 입력.
- 디자인 품질은 루프로 만든다. 코드 생성 한 번으로 전문가 결과가 나오지 않는다. 플랜 → 스크린샷 → 비평 → 수정.
- 예제가 품질의 상한이다. Claude는 예제를 복사·변형하므로, 레퍼런스 앱과 패턴 예제를 최고 수준으로 먼저 만든다.
- 규칙은 문서가 아니라 린터가 지킨다. `@se/eslint-plugin` + `PostToolUse` 훅.

2. 문제 → 설계 원칙

반복되는 문제	설계 원칙	구현 수단
초기 설정을 매번 새로 작성	0분 셋업	<code>create-se-app</code> 템플릿 + <code>/se:new</code>
서비스마다 UI 스타일이 다름	브랜드 코어 고정	<code>@se/tokens</code> , <code>@se/ui</code> , ESLint 강제
그렇다고 다 똑같은 양산형은 싫음	서비스 아이덴티티 필수	<code>/se:identity</code> , <code>se.identity.json</code> , hue 레지스트리
만들어도 "개발자가 만든 티"가 남	디자이너의 작업 순서를 스킴으로	<code>se-design</code> 스킴, 디자인 플랜, 시각 리뷰 루프
웹 경험 적은 구성원의 진입 장벽	요구사항 → 화면 명세 → 코드	<code>/se:spec</code> , <code>/se:page</code> , 6개 페이지 패턴
소수 인원이 전 과정 담당	요구사항~배포를 스킴로 커버	<code>/se:spec</code> → <code>/se:page</code> → <code>/se:api</code> → <code>/se:review</code> → <code>/se:deploy</code>
기존 서비스는 손대기 어려움	점진 도입(strangler)	<code>/se:adopt</code> + <code>codemod</code> , 스타일 격리

3. 아이덴티티 모델 — 같은 가족, 다른 형제 ★

"일관성"과 "개성"은 충돌하지 않는다. 무엇을 고정하고 무엇을 바꾸는지를 명시하면 둘 다 얻는다. (Atlassian의 Jira/Confluence/Trello, Google Workspace의 Docs/Sheets/Slides가 이 모델이다.)

3.1 브랜드 코어 — 모든 SE 서비스에서 동일 (패키지가 강제)

영역	고정 내용
타이포 체계	본문 폰트(Pretendard + IBM Plex Mono), 타입 스케일(12/13/14/16/20/24/32), 행간, 자간 규칙
공간 체계	4pt 리듬, 간격 스케일, 반경 패밀리(4/6/10), 보더·그림자 단계
컴포넌트 해부학	버튼·입력·표·다이얼로그의 형태, 크기, 상태(hover/focus/disabled), 동작
AppShell 골격	좌측 네비 + 상단 헤더, 브레드크럼, ⌘K 커맨드 팔레트, 사용자 메뉴, "SE ▶ 서비스명" 로크업 위치
의미 색	success/warn/danger/info, 작업 상태색(pending/running/succeeded/failed/cancelled) — 액센트와 분리
인터랙션 관례	단축키, 위험 동작 확인, 토스트 위치, 빈/에러/로딩 상태 구조, 모션 시간·이징
카피 관례	한국어 문체 규칙(명사형 버튼, 능동태, 에러=원인+해결)

3.2 서비스 아이덴티티 — 서비스마다 다르게, 반드시 정함

슬롯	선택 범위	예: Job Monitor	예: Dataset Explorer	예: Release Desk
액센트 hue	브랜드 호환 밴드(채도·명도 고정, hue 자유). 레지스트리에서 기존 서비스와 30° 이상, 의미 색(danger≈30°·warning≈70°·success≈155°·info≈260°)과 18° 이상 떨어져야 함 — 가까우면 액센트 칩이 경고/오류처럼 읽힌다(Dataset Explorer에 호박 45°를 줬다가 실제로 겪음)	청록 195°	자두 310°	인디고 235° (초록 150°는 success와 5°라 규칙에 걸림)
뉴트럴 편향	액센트 방향으로 2~3% 기울인 회색	차가운 회색	따뜻한 회색	중성 회색
마크	모노그램 or 아이콘 + 이름 로크업	JM 모노그램	격자 아이콘	태그 아이콘
시그니처 요소 1개	헤더 하단 상태 스트립 / 검색 중심 히어로 / 진행 단계 레일 / 타임라인 리본 등	시스템 상태 스트립	커맨드 팔레트형 검색 히어로	릴리스 단계 레일
기본 밀도	compact / comfortable	compact	compact	comfortable
헤딩 폰트	승인 목록 3~4종(한글 호환) 중 택1	IBM Plex Sans	Space Grotesk 계열 제외, 승인 목록 내	Pretendard 그대로
차트 팔레트	액센트 기반 순차 / 카테고리 변형	액센트 순차	카테고리 8색	액센트 순차
톤	마이크로카피·빈 상태 문구·일러스트 톤	간결·기술적	탐색적·친절	절차적·신중

개성이 나오는 원리: 슬롯이 8개고 각 슬롯의 선택지가 3~8개면, 조합은 수천 가지지만 골격은 하나다. 옆에 두면 "같은 팀 제품"이고, 따로 보면 "다른 제품"이다.

3.3 구현

```
// se.identity.json (프로젝트 루트)
{
  "name": "Job Monitor",
  "mark": { "type": "monogram", "text": "JM" },
  "accent": { "hue": 195 }, // 채도·명도는 브랜드 밴드에서 자동 계산
  "neutralBias": "cool",
  "signature": "status-strip",
  "density": "compact",
  "displayFont": "IBM Plex Sans",
  "chart": "accent-sequential",
  "tone": "terse"
}
```

- @se/tokens 의 createTheme(identity) 가 CSS 변수 세트를 생성. 대비(WCAG AA)와 의미 색 hue 거리는 빌드 시 검증되어 실패하면 빌드 실패.
- 툴킷 저장소의 identities/registry.json 에 서비스별 아이덴티티를 등록. /se:identity 가 hue 충돌·중복 시그니처를 검사한다.
- 시그니처 요소는 @se/ui 의 signatures/ 에 구현된 검증 컴포넌트에서 고른다(자유 발명 아님). 새 시그니처는 승격 절차로 추가.
- 각 프로젝트는 /__identity 개발 전용 라우트에서 자기 아이덴티티 시트(팔레트·마크·타입·시그니처·상태색)를 볼 수 있다.

4. 디자인 품질 — Claude가 웹 디자이너처럼 일하게 하는 장치 ★

"전문가가 만든 것 같다"는 것은 결과물의 속성이 아니라 작업 순서의 속성이다. 디자이너가 하는 일을 순서대로 스킬에 넣는다.

4.1 se-design 스킬 (자동 트리거, se-ui 와 짝)

① 코드 전에 디자인 플랜 (필수, 10줄 이내)

- 이 화면의 *단 하나의 목적*과 사용자가 첫 3초에 봐야 할 것
- 주 액션 1개, 보조 액션, 위험 액션
- 정보 계층 3단계(요약 → 목록 → 상세)와 레이아웃 개념 한 문장
- 밀도(compact/comfortable), 액센트를 쓸 *한 곳*
- 아이덴티티 시그니처를 이 화면에서 어떻게 드러낼지

② 구성 규칙 (디자이너의 기본기)

- 4pt 리듬, 타입 스케일 이탈 금지, 정렬 그리드(레이아웃은 grid/flex + gap, 개별 margin 금지)
- 한 화면에 액센트는 한 곳. 의미 색(success/danger)은 액센트가 아니다
- 모든 블록을 카드로 만들지 않는다. 보더·배경·그림자는 역할별로 하나만 부여
- 숫자는 tabular-nums, 정렬은 소수점 기준. ID·해시·경로는 모노스페이스
- 표는 밀도 우선: 행 높이 32px(compact), 텍스트 좌정렬·숫자 우정렬, 상태는 pill로 형태화
- 요약은 상세보다 위에. KPI 타일은 그 숫자가 화면의 목적일 때만
- 빈 상태에는 *다음 행동*이 있다. 로딩은 최종 형태를 유지한 스켈레톤. 에러는 원인 + 해결
- 모션은 상태 전이에만(150~200ms). 장식 애니메이션 금지
- 카피는 디자인 재료: 사용자 관점 명명, 능동태, 구체성

③ **안티패턴** — "AI가 만든 티"를 금지 이모지 아이콘 / 전부 가운데 정렬 / 그라디언트 히어로 / 모든 블록 동일한 둥근 카드 / 보라-파랑 조합 / 데이터 없는 KPI 타일 나열 / 설명 없는 대시보드 / 화색 100% 뉴트럴 / 내용 없는 "환영합니다" 화면 / 각 섹션에 01·02·03 번호 장식

④ **내부 도구 정보 설계** 스캔되는 화면(읽히는 화면이 아님): 상태를 형태로(pill·스트라이프·점), 필터는 표 바로 위 한 줄, 상세는 드로어/분할, 키보드로 전부 가능, 시간은 상대+절대 병기, 위험 동작은 이름을 다시 입력.

4.2 시각 리뷰 루프 (/se:review)

구현 완료

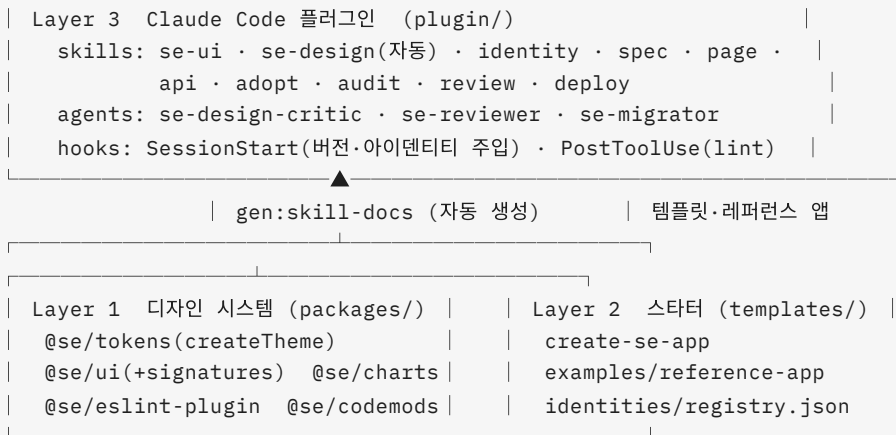
- Playwright: 주요 라우트 스크린샷 (light/dark × 1440/1024 폭, 로딩·빈·에러 상태 강제)
- se-design-critic 에이전트 (이미지 입력) : 루브릭 채점
계층 · 정렬 · 밀도 · 일관성(코어 준수) · 개성(아이덴티티 표현) · 상태 커버리지 · 카피 · 접근성
- 구체 수정 목록 ("필터바와 표 사이 간격 24→12", "KPI 4개 중 2개는 표 헤더로 이동")
- 자동 수정 → 재스크린샷 1회 → docs/design-review.md 기록

- 사람이 하는 "디자인 QA"를 매번 자동으로 한다. 루브릭 점수는 `/se:audit`에 합산된다.
- 비평 에이전트는 읽기 전용이며 수정은 메인 세션이 한다(책임 분리).

4.3 레퍼런스 앱 — 품질의 상한선

`examples/reference-app`: 6개 패턴 전부를 최고 수준으로 구현한 "Job Monitor" 완성본. Storybook·스킬 예제·템플릿의 공통 소스. **Claude가 만드는 모든 화면은 이 앱의 변형**이므로 P1에서 가장 많은 디자인 공수를 여기 쓴다(필요하면 디자이너 1회 참여).

5. 전체 구조 (3개 레이어)



6. Layer 1 — 디자인 시스템 (`packages/`)

6.1 `@se/tokens` — 4계층

`primitive` → **brand (고정)** → **identity (서비스별, createTheme이 생성)** → `semantic (bg.surface, fg.muted, accent.default, status.*)` → `component`.

- W3C DTCG JSON → Style Dictionary → CSS 변수 + Tailwind v4 preset + TS 상수.
- 코드에서는 semantic만 사용(린트). `[data-theme]`, `[data-density]` 속성으로 전환.

6.2 `@se/ui` — 컴포넌트 3티어 + 시그니처

API는 shadcn/Radix 관례를 그대로 따른다(Claude가 아는 이름 → 생성 정확도).

티어	목록
T0 프리미티브	Button, Input, Textarea, Select, Combobox, Checkbox, Switch, RadioGroup, Badge, Tooltip, Popover, Dialog, Sheet, Tabs, Table, Card, Toast, DropdownMenu, Command, Skeleton, Alert, Separator
T1 내부도구 복합	DataTable (TanStack: 서버 페이지네이션·정렬·컬럼 토글·행 선택·CSV), FilterBar, StatusBadge, DescriptionList, JsonViewer, LogViewer(가상 스크롤·ANSI), StatCard, TimeRangePicker, CodeBlock, EmptyState, ErrorState, ConfirmDialog, Form(react-hook-form + zod)
T2 레이아웃/패턴	AppShell, ListDetailPage, DashboardPage, DetailPage, FormWizardPage, SettingsPage
signatures/	StatusStrip, SearchHero, StageRail, TimelineRibbon, MetricMarquee (아이덴티티 시그니처 후보)

각 패턴은 `examples/` 에 로딩·빈·에러 상태를 포함한 완전 예제를 갖는다.

6.3 @se/charts

Recharts 래퍼. 색은 전부 CSS 변수(의미 색은 상태에만, 카테고리는 chart-1..8 고정 순서). 축선 없음·가로 점선 격자·얇은 마크·2개 이상 시리즈는 범례 필수. `ChartCard` (제목·설명·범례·링크), `BarChart` (스택), `LineChart`, `MeterList` (순위·비율), `Sparkline` 은 `@se/ui` 에.

6.4 @se/eslint-plugin

규칙	수준	내용
<code>se/no-raw-color</code>	error	hex/rgb/oklch 리터럴, Tailwind 기본 팔레트·임의값 색 금지 → 토큰 클래스만. <code>className</code> · <code>style</code> · <code>cn()</code> 인자·클래스 변수만 검사(URL 조각·일반 문자열은 안 봄)
<code>se/no-raw-control</code>	error	<code><button></code> <code><input></code> <code><select></code> <code><textarea></code> <code><table></code> 직접 사용 금지 → <code>@se/ui</code> (<code>type="hidden"</code> · <code>file</code> 은 허용)
<code>se/import-from-ui</code>	error	<code>@radix-ui/*</code> · <code>cmdk</code> · <code>sonner</code> · <code>recharts</code> · <code>@tanstack/react-table</code> 직접 import·re-export·동적 import 금지
<code>se/single-accent</code>	warn	한 화면에 동시에 보이는 <code>variant="primary"</code> Button은 하나. 삼항의 양쪽 가지는 하나로 쏜다
<code>se/page-states</code>	warn	<code>DataTable</code> 에 <code>empty</code> 필수, 파일이 원격 데이터를 다룬면(<code>useQuery</code> · <code>isPending</code> 등) <code>loading</code> · <code>error</code> 까지 필수

- `recommended` (앱 코드)와 `library` (`@se/ui` · `@se/charts` 자체 — raw 컨트롤 규칙만 제외) 두 설정. 파서까지 들어 있어 서비스 저장소는 `{ files, ...se.configs.recommended }` 한 줄.
- primitive 토큰은 CSS 변수로 내보내지 않으므로(semantic만 생성) `no-primitive-token` 규칙은 필요 없다.
- 첫 적용에서 실제로 잡은 것: 승인 탭 배너와 헤더에 `primary`가 동시에 보이던 화면, raw `<button>` 사용자 메뉴.

6.5 @se/codemods

jscodeshift 변환(MUI/antd/styled hex → se). 80% 자동, 나머지는 `se-migrator`.

7. Layer 2 — 스타터 (`templates/` , `examples/`)

`npx create-se-app <name>` 또는 `/se:new`.

- Vite + React 19 + TS strict, React Router, TanStack Query, react-hook-form + zod
- AppShell 적용된 `src/app/`, `se.identity.json` (생성 시 `/se:identity` 가 채움), `/__identity` 시트
- **백엔드 무관**: OpenAPI → 타입 클라이언트(`openapi-typescript`), MSW 목 기본 ON. `.env` 의 `VITE_API_BASE` 만 바꾸면 연결
- ESLint(`@se/eslint-plugin`) + Prettier + Vitest + Playwright(스크린샷 리뷰용 설정 포함)
- Dockerfile(multi-stage nginx) + `nginx.conf` + CI 템플릿
- `.claude/settings.json` + `CLAUDE.md` + `docs/spec.md` 템플릿

- **Node 정책 파일 동봉:** `.nvmrc` (22), `package.json` 의 `engines.node: "22.x"` + `packageManager: pnpm@9.x`, `.npmrc` 의 `engine-strict=true` — 생성 즉시 툴킷과 같은 런타임 (§ 11 Node 버전 정책)

8. Layer 3 — Claude Code 플러그인 (plugin/)

구현 상태 (v0.5): 아래 구조가 전부 존재한다 — 스킬 11개(자동 2 + 명령 9), 에이전트 3, 혹은 2, `gen:skill-docs` (43 파일·98 컴포넌트, CI `--check`), `create-se-app` (E2E 검증). `templates/app-vite-react` 는 플레이스홀더 없는 진짜 워크스페이스 앱이라 루트 `typecheck:lint`가 항상 본다. 미완: `@se/*` npm 배포(그 전까지 `/se:new` 는 모노레포 `examples/` 에 생성), `timeline-ribbon`·`metric-marquee` 시그니처, `@se/codemods`.

```
plugin/
├── .claude-plugin/plugin.json      # name: "se", version = @se/ui major.minor
├── skills/
│   ├── se-ui/                   [자동] 컴포넌트·토큰·패턴 지식 (references/ 는 gen:skill-docs 생성)
│   ├── se-design/              [자동] §4.1 디자인 플랜·구성 규칙·안티패턴·내부도구 정보설계
│   ├── se-identity/            /se:identity 서비스 아이덴티티 인터뷰 → 2-3안 미리보기 → se.identity.json
│   ├── se-new/                 /se:new 템플릿 생성 → identity → 첫 실행
│   ├── se-spec/               /se:spec 요구사항 → docs/spec.md (화면·라우트·데이터·상태·디자인 브리프)
│   ├── se-page/               /se:page spec 한 화면 → 디자인 플랜 → 코드 → 스크린샷 1회
│   ├── se-api/               /se:api OpenAPI/기존 코드 → 클라이언트·Query 혹은 MSW
│   ├── se-adopt/             /se:adopt 기존 프로젝트 점진 도입
│   ├── se-audit/             /se:audit 준수율 + 디자인 점수 리포트
│   ├── se-review/            /se:review §4.2 시각 리뷰 루프
│   └── se-deploy/            /se:deploy Dockerfile.nginx·CI
├── agents/
│   ├── se-design-critic.md     # 스크린샷 입력, 루브릭 채점, 읽기 전용
│   ├── se-reviewer.md         # 코드 규칙·상태·a11y, 읽기 전용
│   └── se-migrator.md         # 파일 단위 마이그레이션 실행
├── hooks/hooks.json
│   ├── SessionStart : node -v 검사(§11 정책 미달이면 경고 + .nvmrc 안내) → @se/ui 버전 + se.identity.json 요약 주입
│   ├── PostToolUse(Edit|Write *.tsx) : eslint --fix → 잔여 에러 exit 2 → 즉시 수정
│   └── Stop : 변경 페이지에 상태 누락·액센트 중복 있으면 경고
└── README.md
```

se-identity 스킬 흐름

1. 인터뷰(5문항): 서비스가 하는 일 / 주 사용자와 사용 순간 / 분위기 3단어 / 닮으면 안 되는 형제 서비스 / 가장 중요한 화면
2. `identities/registry.json` 조회 → 사용 가능한 hue 밴드·미사용 시그니처 계산
3. 2-3안 생성: 각 안은 액센트·마크·시그니처·밀도·톤이 다르고, 가장 중요한 화면을 각 안으로 렌더한 미리보기(아티팩트 또는 `/__identity`)
4. 선택 → `se.identity.json` 작성 → 레지스트리 PR

se-ui SKILL.md 골격

```

---
name: se-ui
description: SE 디자인 시스템(@se/ui, @se/tokens)을 쓰는 React 프로젝트에서 UI를 만들거나 수정할 때 반드시 사용.
se-design과 함께 읽는다.
---
## 규칙 (위반 시 린트 실패)
- 색/간격/반경은 semantic 토큰만. hex 금지. 액센트는 `accent.*`만, 서비스 색은 se.identity.json이 정한다.
- 폼 컨트롤·표·다이얼로그는 @se/ui. raw HTML 컨트롤 금지.
- 모든 페이지는 loading / empty / error 상태를 가진다.
- 새 컴포넌트를 만들기 전에 references/components에 있는지 확인. 시그니처 요소는 signatures/에서만.
## 패턴 선택표
| 목록+필터+상세 → ListDetailPage | 지표·상태 요약 → DashboardPage | 단일 객체+탭 → DetailPage | 다단계 입력
→ FormWizardPage | 설정 → SettingsPage |
## 절차
1. se-design의 디자인 플랜을 먼저 쓴다 2. 패턴 예제를 복사한다 3. 변형한다 4. 3상태를 채운다 5. 스크린샷 1회로 확
인한다

```

문서 자동 동기화

packages/ui/src/**/*.tsx (TSDoc + stories) → pnpm gen:skill-docs → plugin/skills/se-ui/references/** → CI에서 diff 검사. 스킬 버전 = @se/ui major.minor, SessionStart 혹은 불일치 경고.

9. 시나리오 A — 신규 프로젝트

```

/se:new job-monitor      → 템플릿 생성 → /se:identity 자동 진입(인터뷰·3안 미리보기·선택) → 첫 실행
/se:spec "잡 목록을 상태별로 보고, 실패 잡을 재시도할 수 있어야 함"
                        → docs/spec.md : 화면 3개, 라우트, 데이터, 상태 전이, 권한, 디자인 브리프
/se:page jobs-list      → 디자인 플랜 → ListDetailPage 변형 + 시그니처(상태 스트립) + 3상태 + MSW
/se:api openapi.yaml     → 클라이언트·Query 혹·목 교체
/se:review              → 스크린샷 → 비평 → 수정 → 재확인 → design-review.md
/se:deploy              → Dockerfile·CI·환경 변수 체크리스트

```

웹 경험이 적은 구성원은 **요구사항 문장 → 아이덴티티 3안 중 선택 → spec 검토** 세 번의 판단만 하면 된다. Figma 없이 spec + 아이덴티티 + 패턴이 디자인 단계를 대체한다.

10. 시나리오 B — 기존 프로젝트 도입 (/se:adopt, strangler)

각 단계가 하나의 PR이고, 어느 단계에서 멈춰도 서비스는 동작한다.

단계	작업	산출물
0 감사	스택· Node·패키지 매니저 버전 ·UI 라이브러리·색/컴포넌트 인벤토리·페이지 목록·스크린샷	docs/adopt-report.md
1 아이덴티티	기존 서비스의 개성(현재 색·톤)을 보존 하며 /se:identity — 형제와 충돌 시에만 조정	se.identity.json
2 기반	Node < 20이면 먼저 Node 업그레이드 PR (CI 이미지·Dockerfile 베이스· .nvmrc · engines 포함, 동작 확인 후 진행). 그다음 @se/tokens 설치, @layer se 로 격리, ThemeProvider, 폰트. 시각 변화 0	(Node PR) + 기반 PR
3 셸	AppShell로 네비·헤더 교체 + 시그니처 요소 적용	첫 눈에 보이는 일관성+개성
4 페이지	페이지 단위 codemod → Claude 보정 → 전/후 스크린샷 비평. 트래픽 많은 순	페이지별 PR
5 강제	eslint warn→error, 혹 ON, 잔여 라이브러리 제거, Node 22로 통일 + engine-strict	완료

11. 가드레일 · 거버넌스

- **기계적 강제:** ESLint(§ 6.4) + PostToolUse 혹. **시각적 강제:** `/se:review` 루브릭 점수 하한(예: 80/100) 미만이면 PR 체크 실패 (선택).
- **에이전트 역할 분리:** critic·reviewer는 읽기 전용, 수정은 메인 세션.
- **아이덴티티 레지스트리:** 서비스별 hue·시그니처·마크 등록. 신규 서비스는 충돌 검사 통과 필수. 형제와 너무 닮으면 `/se:identity` 가 거부.
- **승격 규칙:** 로컬 컴포넌트/시그니처가 2개 서비스에서 반복되면 `@se/ui` 로 승격(TSDoc:story-example 필수).
- **버전·업그레이드:** § 15. 툴킷은 한 버전으로 움직이고 앱은 태그에 고정, 받는 절차는 `/se:upgrade`.
- **문서:** Storybook(사람용) = 스킬 레퍼런스와 같은 소스. 아이덴티티 갤러리(모든 서비스 시트 나열)로 가족 초상화를 유지.

Node 버전 정책

기존 서비스마다 Node 버전이 제각각인 것은 "내 컴퓨터에서는 되는데"의 주범이다. 툴킷은 버전을 **고정**하고, 도입은 **하한선**으로 관리한다.

대상	규칙	강제 수단
툴킷 저장소·플러그인·레퍼런스 앱	Node 22 LTS 고정 , pnpm 9	<code>.nvmrc</code> , <code>engines.node: "22.x"</code> , <code>.npmrc engine-strict=true</code> , <code>packageManager</code> 필드(corepack)
신규 서비스 (<code>/se:new</code>)	툴킷과 동일	템플릿에 위 파일 동봉
기존 서비스 (<code>/se:adopt</code>)	도입 중 하한 20.19 (Vite 7·Tailwind v4 최소), 완료(5단계) 시 22로 통일	0단계 감사 항목, 20 미만이면 기반 단계 전에 업그레이드 PR
모든 세션	요구 버전 미달이면 시작 시 경고	SessionStart 혹 <code>node -v</code> 검사 → <code>.nvmrc / nvm use</code> 안내, 설치·빌드 명령 실행 전 재확인

- 왜 22인가: 툴체인(Vite 7, Tailwind v4, MSW 2)이 20.19+를 요구하고, 22는 2027-04까지 LTS. 20은 2026-04에 EOL이라 신규 기준이 될 수 없다.
- 버전 올림은 툴킷에서 한 번에: `.nvmrc·engines` 변경 → 플러그인 major 업 → SessionStart 혹이 각 서비스에 알림.
- 예외(레거시 런타임을 못 올리는 서비스)는 도입 대상에서 제외하고 감사 리포트에 사유를 남긴다.

12. 저장소 구조

```
se-web-toolkit/
├── packages/ tokens · ui · charts · eslint-plugin · codemods · create-se-app
├── templates/app-vite-react/
├── examples/ reference-app · patterns/*
├── identities/ registry.json · <service>.identity.json
├── plugin/ (§8)
├── apps/storybook/ (+ 아이덴티티 갤러리)
├── scripts/ gen-skill-docs.ts · check-identity.ts
├── .claude-plugin/marketplace.json
├── CHANGELOG.md (세 칸: 바뀐 것 / 화면 변화 / 앱에서 할 일)
└── docs/DESIGN.md
```

설치: `/plugin marketplace add <org>/se-web-toolkit` → `/plugin install se@se-web-toolkit`

13. 로드맵

단계	범위	완료 기준
P1 기반+품질 (완료)	tokens(4계층, createTheme), 컴포넌트 50개(98 export), 시그니처 3종, 레퍼런스 앱 3개(모니터링·조회·업무 처리), 템플릿+CLI, se-ui · se-design 스킴, 명령 9개, 혹은 2, eslint-plugin, charts	세 시나리오 원본 확보. 디자인 리뷰 56 → 70/80
P2 검증+도입	새 세션에서 /se:new 로 4번째 서비스를 스킴만으로 생성해 품질 검증, /se:review 루프 실행, /se:adopt 로 기존 서비스 1개, codemods, 나머지 시그니처 2종	기존 서비스 1개 5단계 완료. 스킴이 만든 화면이 70/80 이상
P2.5 운영 (완료)	버전 한 값·CHANGELOG·태그 자동·CI check:release, 앱은 태그 고정, /se:upgrade, /se:audit 버전 비교 (\$15)	툴킷 변경이 앱에 닿는 경로가 한 가지
P3 배포	@se/* npm 배포(→ 모노레포 밖 npx create-se-app), 시그니처 2종 추가, Storybook·아이덴티티 갤러리, 기여 가이드, 자동 업그레이드 PR	웹 비전공 구성원이 혼자 서비스 1개 출시

성공 지표: 셋업→첫 화면 배포 시간 / se:audit 준수율·디자인 점수 / 서비스 간 hue·시그니처 중복 0 / 신규 구성원 첫 PR까지 시간.

14. 핵심 결정과 이유

결정	대안	선택 이유
아이덴티티 2층(코어 고정 + 서비스 슬롯 필수)	단일 테마 / 완전 자유	단일 테마 = 양산형, 완전 자유 = 일관성 붕괴. 슬롯 모델이 유일하게 둘을 만족
시그니처 요소를 검증된 목록에서 선택	매번 자유 발명	발명은 품질 편차가 크다. 목록은 성장시킬 수 있다(승격 규칙)
디자인 플랜 + 스크린샷 비평 루프	코드 규칙만	코드 규칙은 "틀리지 않음"을 보장할 뿐 "잘 만들"을 보장하지 않는다
레퍼런스 앱에 디자인 공수 집중	컴포넌트만 잘 만들기	Claude는 예제를 변형한다. 예제 품질 = 결과 품질
npm 패키지 배포	shadcn식 소스 복사	문제의 본질이 "갈라짐". 개성은 토큰 슬롯으로, 코드 분기로가 아니라
shadcn/Radix API 관례	독자 API	Claude가 이미 아는 이름 → 생성 정확도
Tailwind v4 + CSS 변수 토큰	CSS Modules / CSS-in-JS	생성 품질 최상, 토큰이 CSS 변수라 기존 프로젝트에도 주입 가능
문서를 코드에서 생성	손으로 쓴 스킴 문서	스킴 드리프트 차단
규칙을 린터로	긴 가이드 문서	LLM은 문서를 잊지만 exit code는 못 잊는다
백엔드 무관(OpenAPI + MSW)	백엔드 표준화	백엔드는 비표준이 현실. UI 먼저 완성 후 연결
Node 22 LTS 고정 + engine-strict	범위 허용(>=18) / 프로젝트 별 자율	툴체인 하한이 20.19이고 버전 혼재가 재현 불가 버그의 주범. 고정 + 혹검사가 가장 싸다

리스크

- 시그니처·hue 슬롯이 부족해질 때 → 슬롯 확장(그라데이션 아님, 새 시그니처 승격·hue 밴드 세분화).
- 비평 루프 비용(시간·토큰) → se:page 는 스크린샷 1회, 전체 루프는 se:review 에서만.
- 레퍼런스 앱 품질 확보 → P1에 디자이너 1회 참여 또는 팀 내 디자인 감각 있는 인원이 최종 승인.
- 기존 서비스 UI 라이브러리 혼재 → codemod 80% + se-migrator.
- 기존 서비스 Node 버전 상이(18 이하) → 0단계 감사에서 드러나고, 기반 단계 전 별도 업그레이드 PR로 분리. 못 올리면 도입 제외.

15. 운영 — 툴킷이 바뀌면 앱은 어떻게 만나

배포된 서비스가 생긴 뒤의 질문은 "툴킷을 고치면 서비스들은 어떻게 되나"다. 답은 아무것도 저절로 바뀌지 않는다이고, 받는 경로를 한 가지로 고정한다.

15.1 툴킷 쪽 — 한 버전, 한 기록

- `@se/tokens · ui · charts · eslint-plugin · create-se-app` 플러그인은 **한 버전**으로 움직인다(루트 `package.json` 이 기준, `pnpm release:bump x.y.z` 로 한 번에). 따로 올리면 조합이 폭발한다.
- semver 의미를 고정한다: **patch** = 화면 변화 없음 · **minor** = 추가 또는 화면 변화, 코드 수정 불필요 · **major** = 앱 코드를 고쳐야 함.
- `CHANGELOG.md` 는 항목마다 세 칸 — 바뀐 것 / 화면 변화 / **앱에서 할 일**. 앱 담당자는 세 번째 칸만 읽는다.
- `CI check:release: packages/ · templates/ · plugin/` 이 바뀐 PR 은 버전이 main 과 달라야 하고 `CHANGELOG` 에 그 버전 항목이 있어야 한다. merge 되면 `v<버전>` 태그가 자동으로 찍힌다.
- 컴포넌트 변경은 레퍼런스 앱 3개에서 라이트·다크·1024 스크린샷으로 확인한 뒤 merge 한다(한 컴포넌트가 세 서비스 모두에서 갱신되는가).

15.2 앱 쪽 — 태그 고정, 받을 때 `/se:upgrade`

- 앱의 `@se/*` 참조는 태그다: `github:bbora-min/se-web-toolkit#v0.5.0&path:packages/ui`. `/se:new` 가 CLI 와 같은 버전으로 고정하고, `/se:adopt` 2단계는 최신 태그를 고른다.
- `/se:upgrade`: `CHANGELOG` 의 "앱에서 할 일" 을 먼저 보여 줌 → 참조를 새 태그로 → `typecheck · lint` → 전/후 스크린샷 픽셀 diff 로 **달라진 화면 목록** → 그 화면만 `/se:review` → 전/후 이미지가 붙은 PR. 사람은 PR 만 본다.
- `/se:audit` 가 "이 앱은 v0.5.0, 최신 v0.7.0, 밀린 '앱에서 할 일' 2건" 을 보여 준다.
- 플러그인(스킬·훅)은 `claude plugin update` 로 따로 받되 같은 버전으로 맞춘다 — 규칙과 컴포넌트가 일치해야 한다.

15.3 호환 정책

- 깨지는 변경은 한 minor 동안 옛 방식을 남기고 콘솔 경고(deprecation). 앱은 두 버전 안에 옮긴다.
- 토큰 이름·컴포넌트 prop 은 계약이다. 바꿀 땐 `codemod` 를 같은 PR 에 넣는 것이 조건(P3 `@se/codemods`).
- 지원 범위는 최신 major 와 직전 major. 그보다 오래된 앱은 `/se:audit` 가 "업그레이드 필요" 로 표시한다.

15.4 아직 안 한 것

- 레퍼런스 앱 스크린샷의 CI 기준 이미지 비교("화면이 바뀌었는데 patch" 를 기계로 잡기).
- 툴킷 릴리스 때 각 앱 리포에 업그레이드 PR 을 자동으로 여는 것 — 두 번째 앱이 생기면.
- npm 배포(그때 태그 대신 semver 범위).